

Trace-Based Contrastive Learning for Static Analysis Alarm Ranking Without Feature Engineering



이정훈

Programming System Laboratory

Motivation

- 코드 재사용, 리팩토링, 유사 구현 과정에서 과고 발생한 취약점이 비슷하게 다시 발생
Recurring Vulnerability 혹은 재발하는 취약점
- 정적분석을 통해 프로그램을 실행하지 않고 취약점 탐지 가능, 하지만 알람이 매우 많이 생성, 그중 허위경보도 포함 → 개발자가 모두 다 확인하기는 어려움
- Recurring vulnerability 기반 랭킹: 기존에 확인된 취약점과 유사한 새 알람을 더 위험할 가능성이 높은 알람부터 내림차순 배치 → 실제 취약점일 가능성이 높은 알람을 먼저 확인
- Tracer - 에러 종류별로 수동 설계된 피처 사용. 우리는 LLM을 써서 수동 피처설계 없이 알람 랭킹을 하고자 함.

Prior study :Tracer

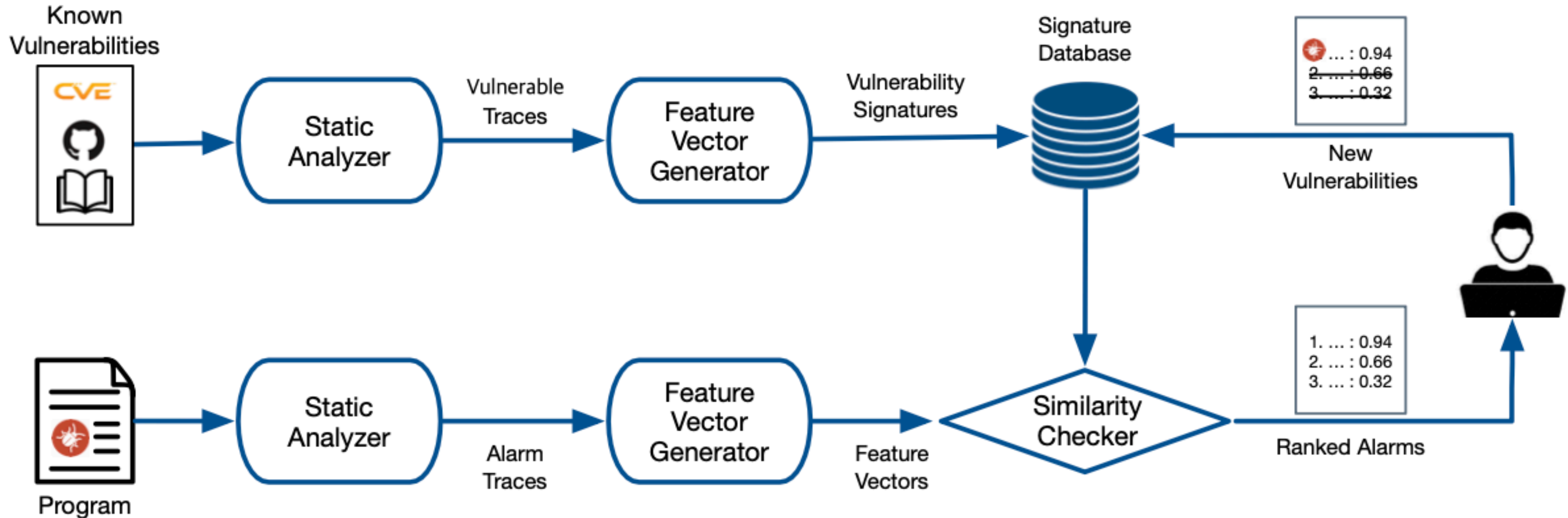


Figure 2: System overview of TRACER

Static analyzer

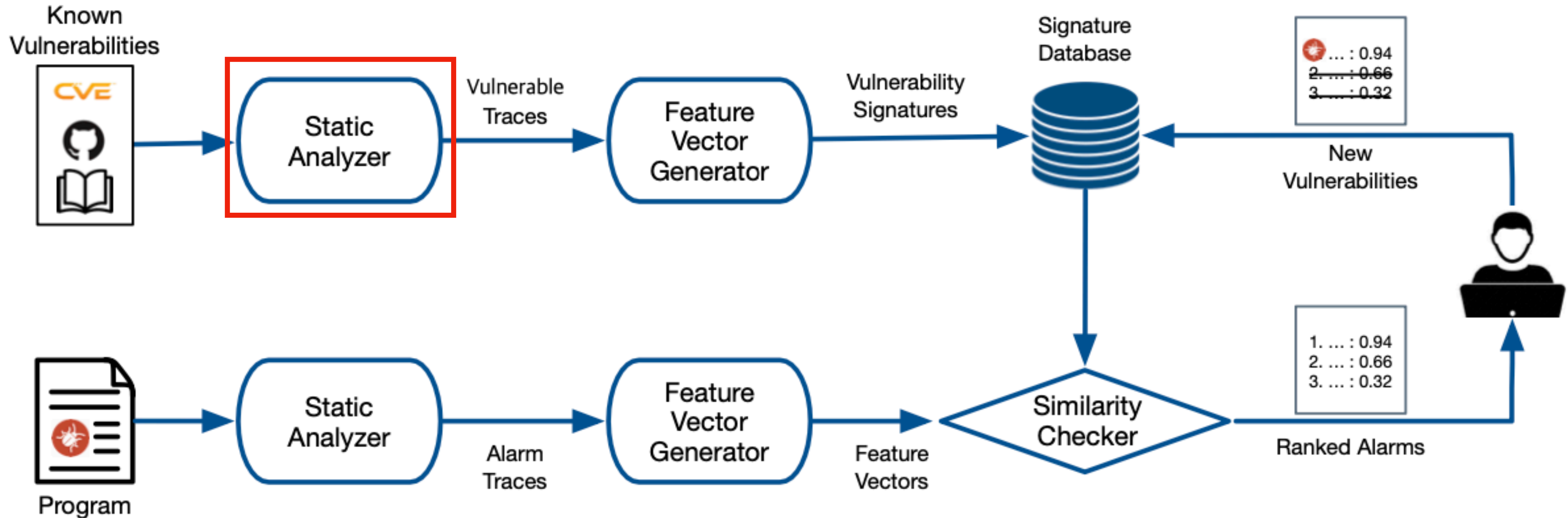


Figure 2: System overview of TRACER

Using handmade features

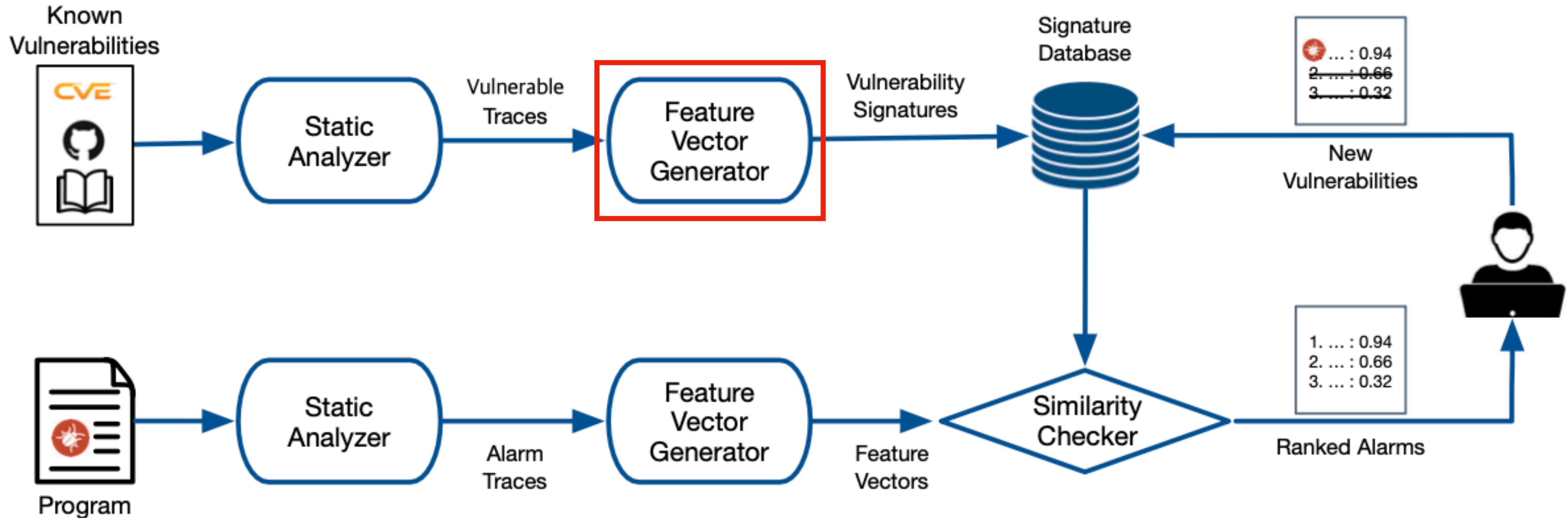


Figure 2: System overview of TRACER

Tracer Feature-Low level

Table 1: Features of traces. E and K represent an arbitrary expression and a constant, respectively.

Name	Description
NumOfOpX	# primitive operator X on the trace
NumOfLibX	# calls to library X on the trace
LargerThanConst	# expressions of the form $E > K$ or $E \geq K$
SmallerThanConst	# expressions of the form $E < K$ or $E \leq K$
EqualToVar	# expressions of the form $E == K$
NotEqualToVar	# expressions of the form $E != K$
EqualToPercentage	# expressions of the form $E == \text{'\%'}$

Tracer Feature-High level

Table 1: Features of traces. E and K represent an arbitrary expression and a constant, respectively.

Name	Description
NumOfOpX	# primitive operator X on the trace
NumOfLibX	# calls to library X on the trace
LargerThanConst	# expressions of the form $E > K$ or $E \geq K$
SmallerThanConst	# expressions of the form $E < K$ or $E \leq K$
EqualToVar	# expressions of the form $E == K$
NotEqualToVar	# expressions of the form $E != K$
EqualToPercentage	# expressions of the form $E == \text{'\%'}$

← 외부 입력의 값이 범위에 맞는지

Tracer Feature-High level

Table 1: Features of traces. *E* and *K* represent an arbitrary expression and a constant, respectively.

Name	Description
NumOfOpX	# primitive operator X on the trace
NumOfLibX	# calls to library X on the trace
LargerThanConst	# expressions of the form $E > K$ or $E \geq K$
SmallerThanConst	# expressions of the form $E < K$ or $E \leq K$
EqualToVar	# expressions of the form $E == K$
NotEqualToVar	# expressions of the form $E != K$
EqualToPercentage	# expressions of the form $E == \text{'\%'}$

← 특정 값과 일치하거나 일치하지 않는지

Tracer Feature-High level

Table 1: Features of traces. *E* and *K* represent an arbitrary expression and a constant, respectively.

Name	Description
NumOfOpX	# primitive operator X on the trace
NumOfLibX	# calls to library X on the trace
LargerThanConst	# expressions of the form $E > K$ or $E \geq K$
SmallerThanConst	# expressions of the form $E < K$ or $E \leq K$
EqualToVar	# expressions of the form $E == K$
NotEqualToVar	# expressions of the form $E != K$
EqualToPercentage	# expressions of the form $E == \text{'\%'}$

← 위험한 %문자 (%x) 가 들어있는가

Tracer pipeline

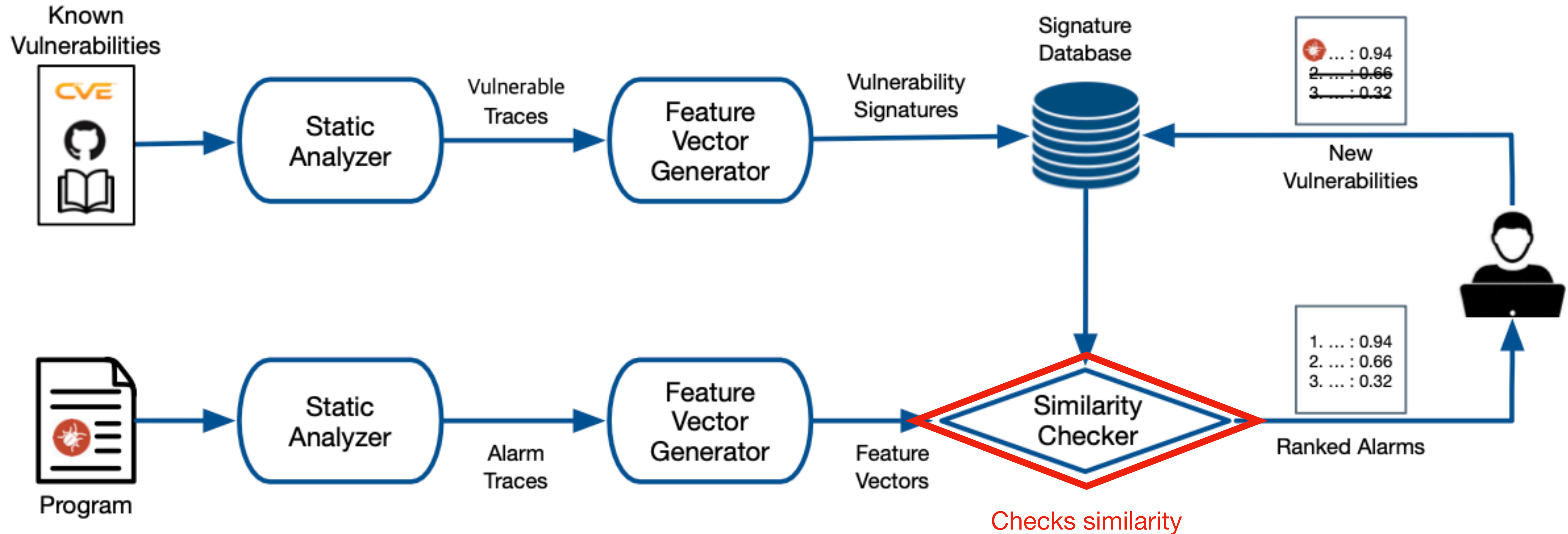
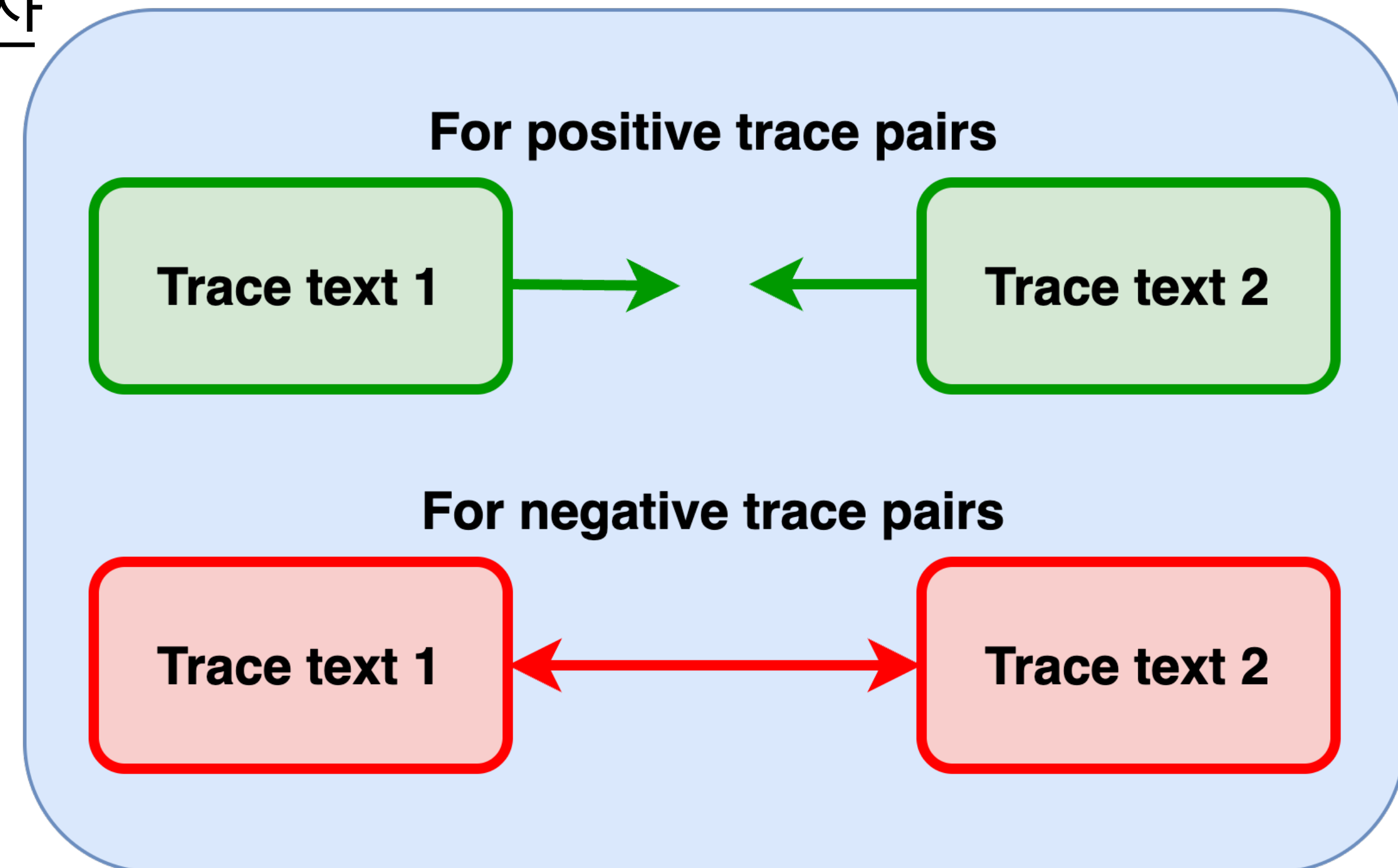


Figure 2: System overview of TRACER

Our Key Idea

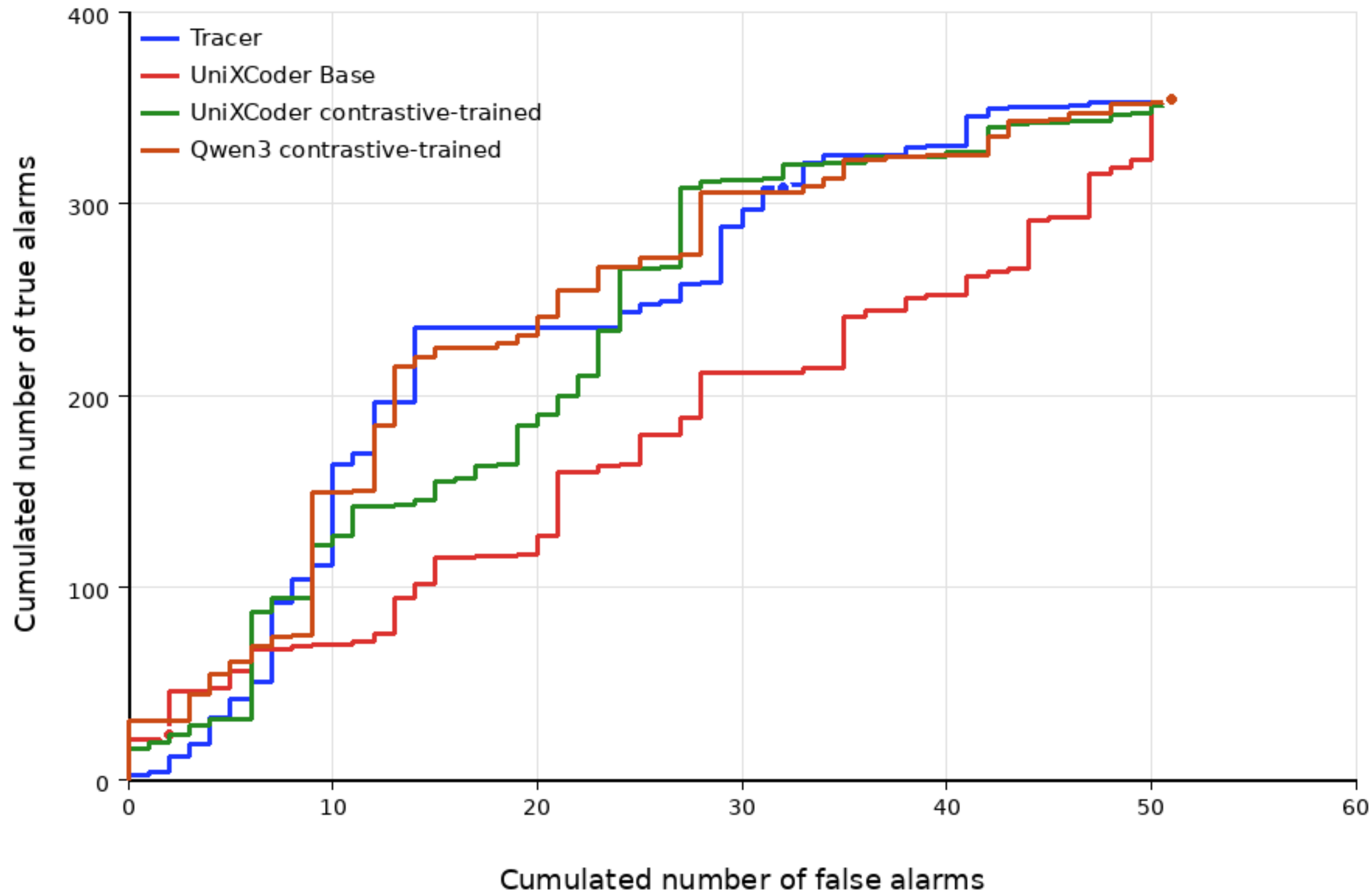
대조학습은 유사한 데이터는 embedding space에서 가깝게,
유사하지 않은 데이터는 멀게 배치하도록 encoder를 미세조정(finetuning)

각 코드흐름을 Encoder only LLM을 사용해서 embedding 생성
두 코드흐름의 embedding의 cosine 유사도 계산
Positive pair는 높아지도록, Negative pair는
margin보다 작아지도록 finetuning



Results

결과는 ROC그래프의 아래 면적인 AUC로 점수를 측정. 1에 가까울수록 좋다



Results

모델	AUC score
Tracer	0.681
VulBERTa	0.554
UniXcoder(base)	0.495
Qwen3 0.6b (base)	0.621
UniXcoder(contrastive learning)	0.651
Qwen3 0.6b (contrastive learning)	0.691



End